Optical Flow

Dr. Praisan Padungweang, Collage of Computing, KKU

Credit: Steve Elston

HARVARD
Extension School

Optical Flow

Optical flow algorithms track image-to-image motion

- Optical flow fields provide **pixel-by-pixel (dense) motion** from video
- Optical flow algorithms have many applications:
  - Autonomous vehicles
  - Robotics
  - Action recognition
  - Augmented reality
  - Encoding and deblurring video
  - ....
- Like many areas of CV, optical flow is being revolutionized by deep NN algorithms

Optical Flow

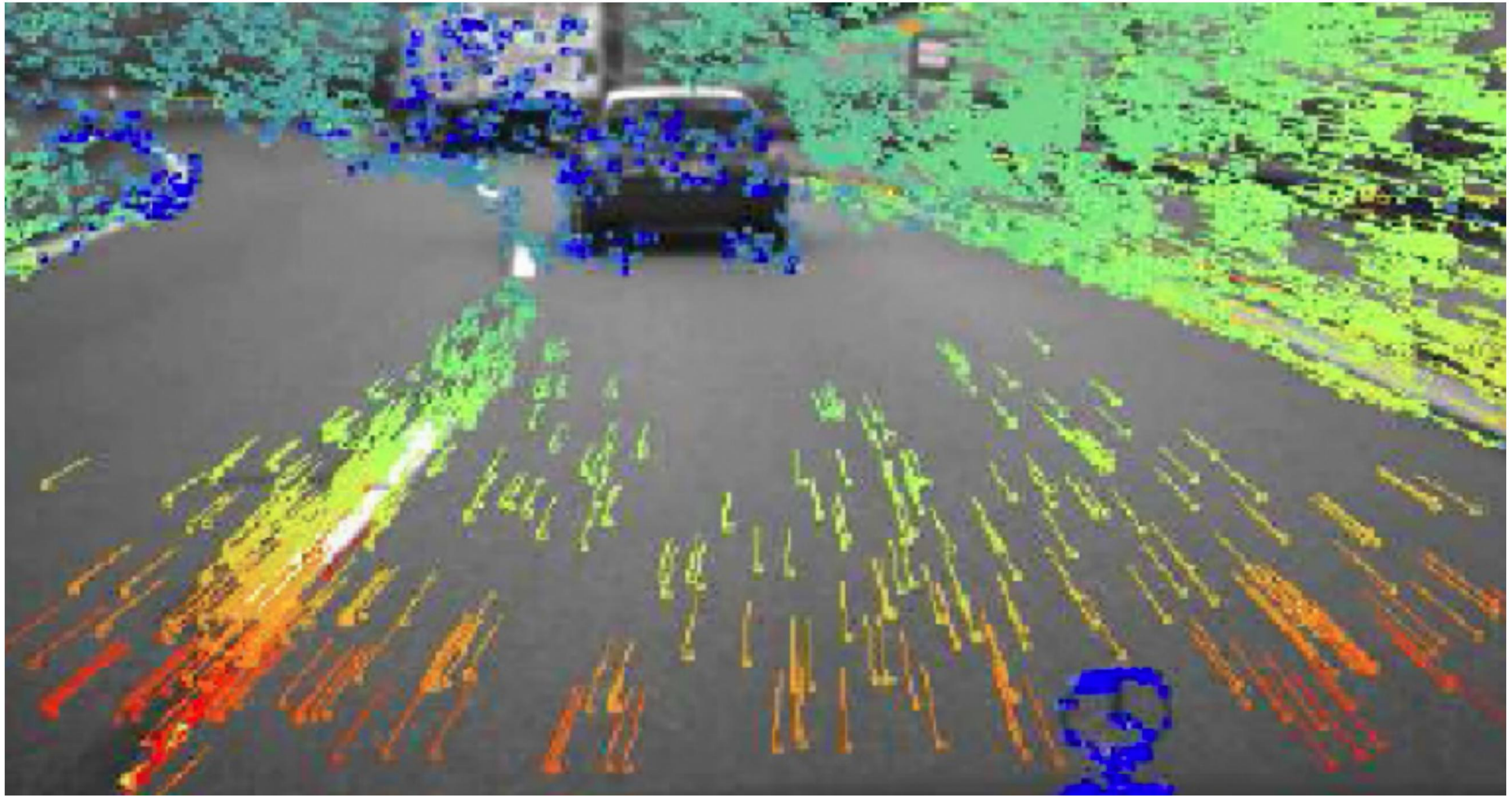
Optical flow algorithms track image-to-image motion

Cred Figure 5. Results on KITTI 2015 *training* and *test* sets. Fine-tuning fixes large regions of errors and recovers sharp motion boundaries.

TensorFlow implementation

Optical Flow

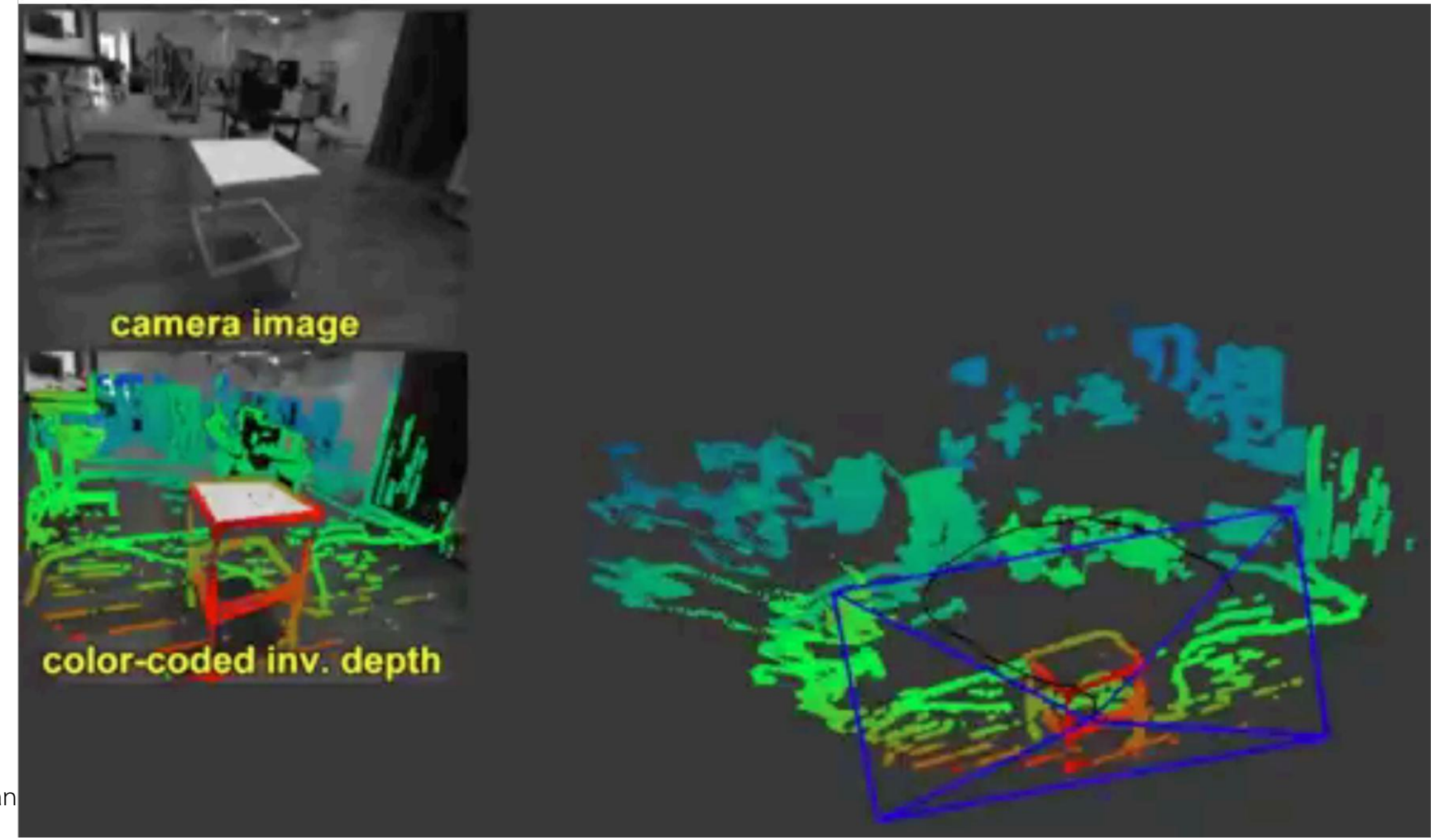
Optical flow field estimation for autonomous vehicles



Credit, Kris K

Optical Flow

Optical flow algorithms for mobile robot scene estimation



Credit, Kris Kitan

Optical Flow

COLLEGE OF COMPUTING
KHON KAEN UNIVERSITY

Overview of lesson

- What is the relationship between optical flow and projective transforms?
- Small movement, consistency assumptions, and the iterative Lucas-Kanade (iLK) algorithm
- Dealing with multiple scales
- Regularization with the Horn-Schunck algorithm
- Improving performance with the LV-L1 algorithm
- Evaluation of optical flow algorithms
- Deep NN algorithms for optical flow

Optical Flow vs. Projective Transformation

COLLEGE OF COMPUTING
KHON KAEN UNIVERSITY

Compare optical flow models to projective transform models

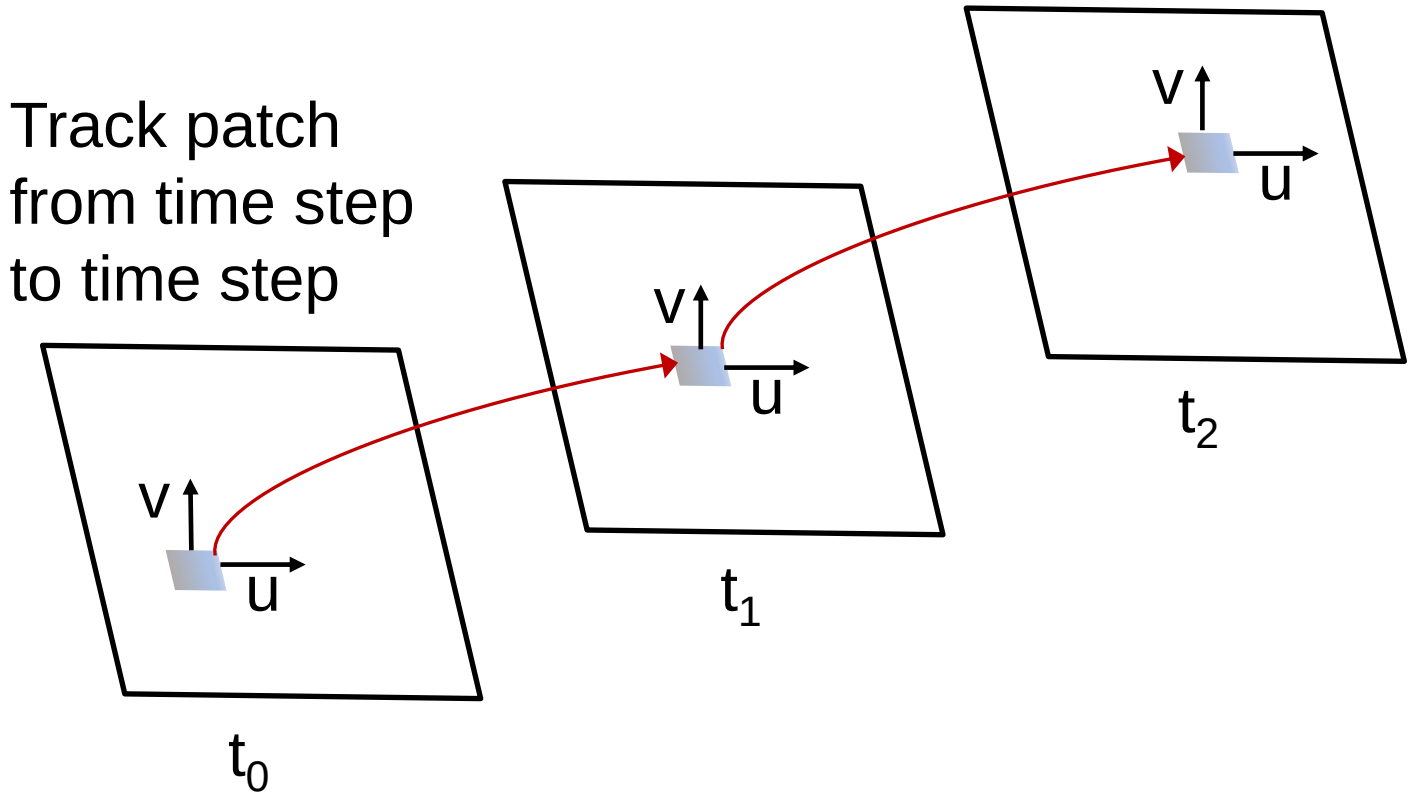
- Both algorithms use image warping
- Parameters for projective transformation models are estimated globally
  - A single transform is applied to the entire image
  - Cannot localize motion
- Parameters for optical flow models are estimated pixel-by-pixel
  - Is a **dense learning task**
  - Warping on a pixel-by-pixel basis
  - Localizes motion

THE SMALL MOTION CONSISTENCY ASSUMPTIONS

Small Motion and Consistency Assumptions

Assumptions about motion in video simplify a flow model

- Frame to frame **motion is small**
- Frame to frame **illumination and color are constant**



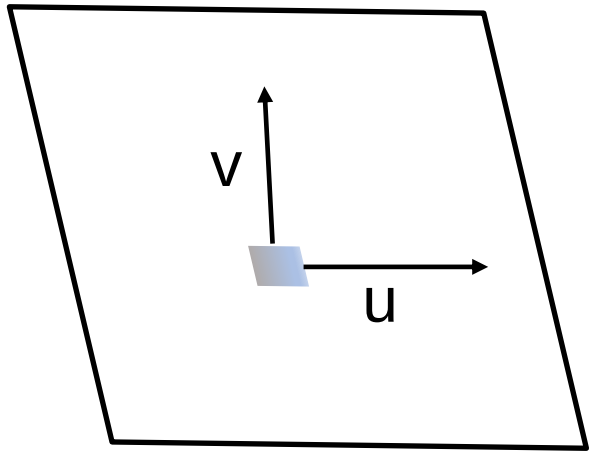
Small Motion and Consistency Assumptions

Assumptions about motion in video simplify flow model

- Velocity in the x and y directions

$$u = \frac{\partial x}{\partial t}$$

$$v = \frac{\partial y}{\partial t}$$



- Estimate $[u, v]$, the **optical flow parameters**
- Goal of models is to estimate optical flow field for time sequenced images, e.g. video

Small Motion and Consistency Assumptions

Assumptions about motion in video simplify flow model

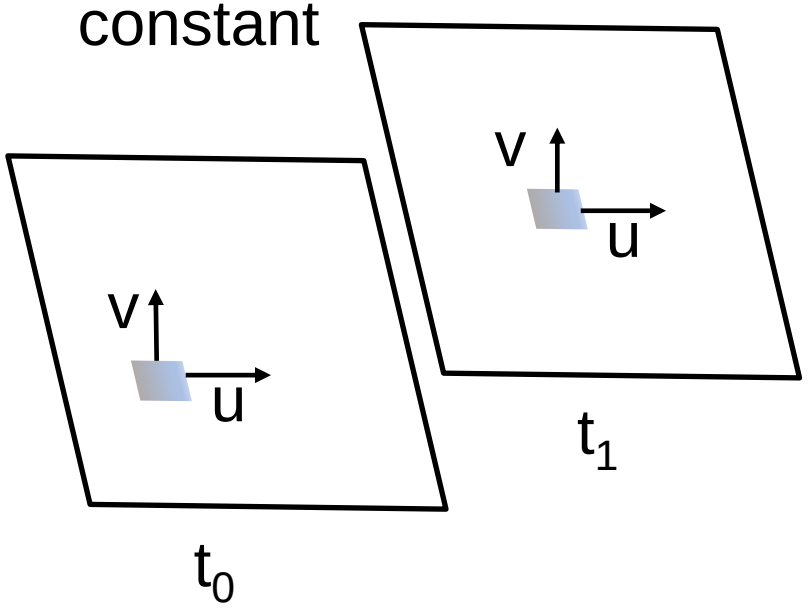
- Small time step, $\delta t \longrightarrow$ small motion from frame-to-frame

$$\delta t = t_1 - t_0$$

$$\delta x = u\delta t$$

$$\delta y = v\delta t$$

Illumination in patch remains constant



Small Motion and Consistency Assumptions

Assumptions about motion in video simplify flow model

- Illumination, *I(x,y)*, is constant for a small time step, δt

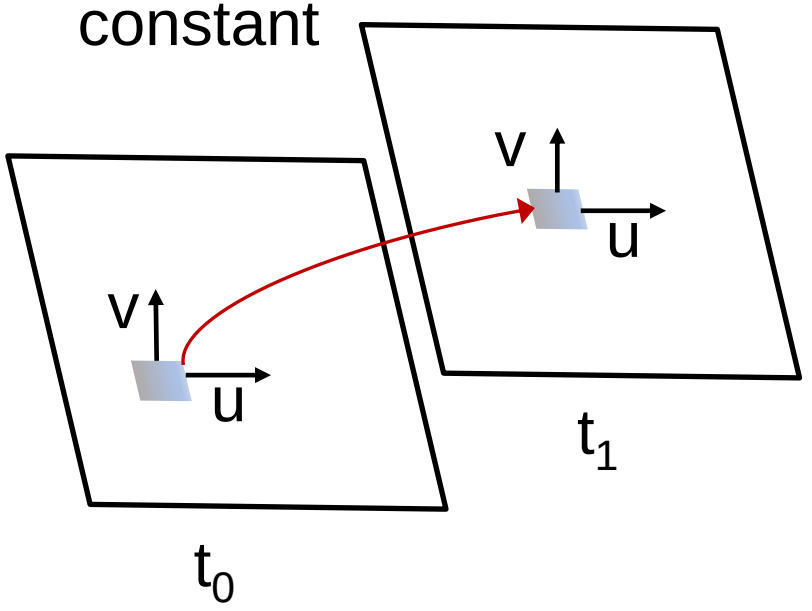
$$I(x, y, t) = I(x + u\delta t, y + v\delta t, t + \delta t) = Const$$

Or,

$$\frac{dI(x(t), y(t), t)}{dt} = 0$$

- We call this **brightness consistency**

Illumination in patch remains constant



THE LUCAS-KANADE ALGORITHM

The Lucas-Kanade Algorithm

For small change in position over small time step, δt, we can write the intensity relationship:

$$I(x + u\delta t, y + v\,\delta t, t + \delta t) = I(x, y, t)$$

- Need this relationship in a **linear form** we can work with
- Recall the **Tylor expansion**, about current position

$$small\ change\ in\ position = [\delta x = x - x', \delta y = y - y']$$

$$f(x, y) \approx f(x', y') + \frac{df(x', y')}{dx}(x - x') + \frac{df(x', y')}{dy}(y - y')$$

- Taylor expansion for the intensity relationship:

$$I(x, y, t) = I(x', y', t) + \frac{dI}{dx}\delta x + \frac{dI}{dy}\delta y + \frac{dI}{dt}\delta t$$

The Lucas-Kanade Algorithm

For small change in position over small time step, δt, we can write the intensity:

$$I(x + u\delta t, y + v\,\delta t, t + \delta t) = I(x, y, t)$$

- Taylor expansion for the intensity relationship:

$$I(x, y, t) = I(x', y', t) + \frac{dI}{dx}\delta x + \frac{dI}{dy}\delta y + \frac{dI}{dt}\delta t$$

- Subtracting the $I(x, y, t)$ term from both sides:

$$\frac{dI}{dx}\delta x + \frac{dI}{dy}\delta y + \frac{dI}{dt}\delta t = 0$$

- Finally dividing both sides by δt and letting $\delta t \longrightarrow 0$:

$$\frac{dI}{dx}\frac{dx}{dt} + \frac{dI}{dy}\frac{dy}{dt} + \frac{dI}{dt} = 0$$

The Lucas-Kanade Algorithm

For small change in position over small time step, δt, we can write the intensity relationship:

$$\frac{dI}{dx}\frac{dx}{dt} + \frac{dI}{dy}\frac{dy}{dt} + \frac{dI}{dt} = 0$$

- Does this look familiar? – recall the **chain rule of calculus!**
- In matrix-vector notation we can write this relation:

$$\nabla I^T \boldsymbol{v} + \frac{dI}{dt} = 0$$

Where the **gradient of image intensity,** and **velocity** vectors:

$$\nabla I^T = \begin{bmatrix} \frac{dI}{dx} \\ \frac{dI}{dy} \end{bmatrix}, \qquad \boldsymbol{v} = \begin{bmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix}$$

Finding spatial gradients

Recall we can find a spatial gradient with a Laplacian operator

$$\mathcal{L}(x,y) = \frac{\partial\, I(x,y)}{\partial x} + \frac{\partial\, I(x,y)}{\partial y}$$

- Simple Laplacian kernel:

$$K_{\mathcal{L}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- Laplacian kernel with less directional bias:

$$K_{\mathcal{L}} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

The Lucas-Kanade Algorithm

We need to solve the linearized equations for the optical flow, $[u, v]$:

$$\frac{dI}{dx}\frac{dx}{dt} + \frac{dI}{dy}\frac{dy}{dt} + \frac{dI}{dt} = 0$$

$$\frac{dI}{dx}u + \frac{dI}{dy}v + \frac{dI}{dt} = 0$$

- There are **2 unknowns**, $[u, v]$, but only **1 equation**!
- Can solve by using more cases
  - Use *x*, *y* locations from say a 5×5 patch
  - Now 25 equations
  - Find by **least squares solution**
- This approach enforces a **constant motion constraint** on the solution
  - We assume all 25 pixels move with the same flow

The Lucas-Kanade Algorithm

We need to solve the linearized equations for the optical flow, $[u, v]$:

$$\frac{dI}{dx}u + \frac{dI}{dy}v + \frac{dI}{dt} = 0$$

- Using the more compact notation of I_x, I_y, I_t for the spatial and temporal derivatives of the image intensity we can write a linear system of 25 equations:

$$I_x(p_1)u + I_y(p_1)v = -I_t\ (p_1)$$
$$I_x(p_2)u + I_y(p_2)v = -I_t\ (p_2)$$
$$\vdots$$
$$I_x(p_{25})u + I_y(p_{25})v = -I_t\ (p_{25})$$

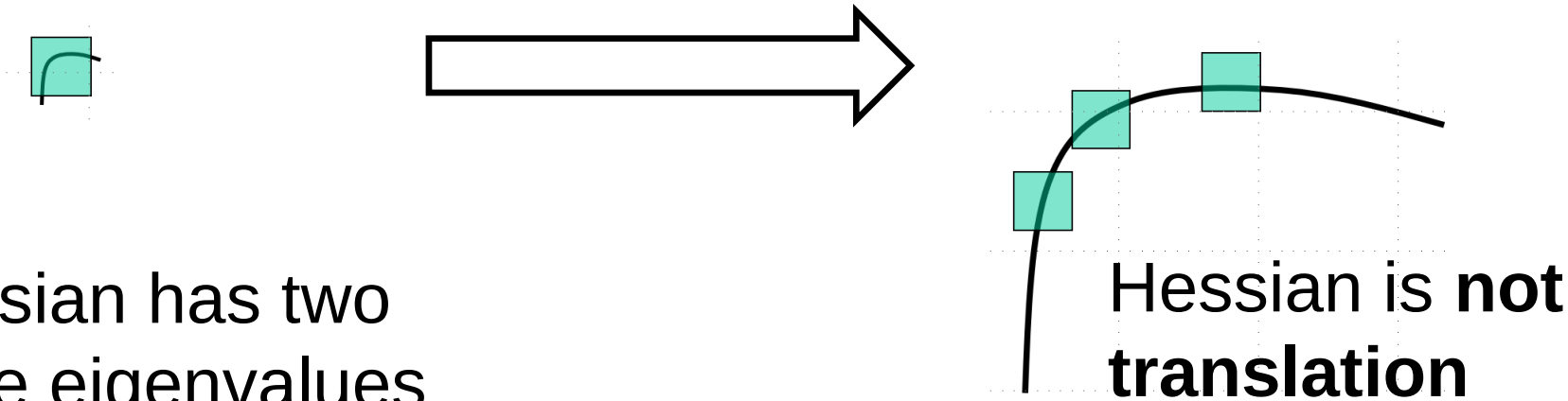
- Then find **least squares estimates** for the optical flow, $[u, v]$, at center of patch, $[x, y]$

DEALING WITH SCALE

Problem with scale

How does the gradient change with feature scale

- Gradient is not scale independent



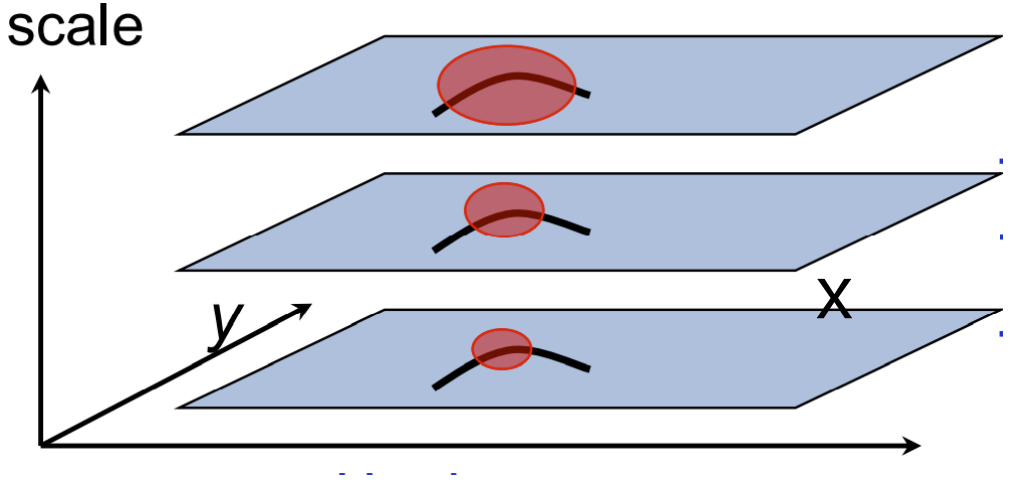
Hessian has two large eigenvalues

Hessian is **not translation invariant, no inverse!**

Multi-Scale Feature Identification

Pyramid structure enables extraction of interest points at several scales

- Pyramid scales from fine to coarse in octaves
- Features at each scale extracted
- Allow flow to be computed at several scales
- Small motion is scale dependent, so get wider range of velocities



ITERATIVE LUCAS-KANADE ALGORITHM

Iterative Lucas-Kanade algorithm

COLLEGE OF COMPUTING
KHON KAEN UNIVERSITY

We can apply the **iterative Lucas-Kanade (ilk)** algorithm

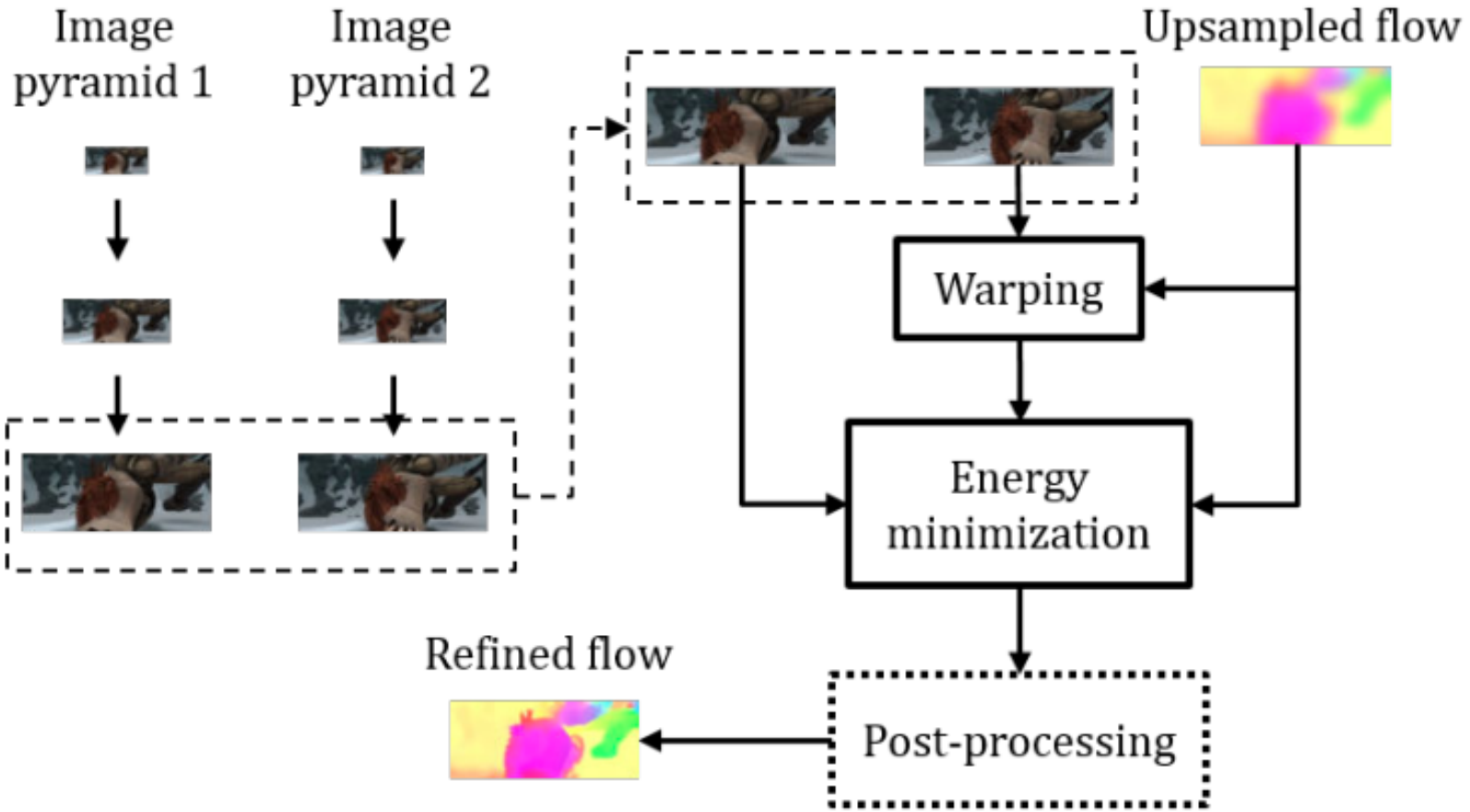
- ilk applies the Lucas-Kanade algorithm large to small scale iteratively

```
Procedure iLK(image1, image2):
    pyramid1 = LaplacianPyramid(image1)
    pyramid2 = LaplacianPyramid(image2)
    flow = zeros(pyramid.shape)
    for scale in pyramid: # Loop from coarse to fine
        flow = UpSample(flow) # Up sample flow to scale
        warped_image = Warp(pyramid2(scale), flow) # Dense warp
        # flow components updated for each scale
        flow = LucasKanade(pyramid1(scale), warped_image) + flow
    Filter(flow) # Post processing
    return flow
```

Iterative Lucas-Kanade algorithm

We can apply the **iterative Lucas-Kanade (iKL)** algorithm

- iLK applies the Lucas-Kanade algorithm large to small scale iteratively



Credit: Sun, et. al, 2018

DEEP OPTICAL FLOW ALGORITHMS

Deep Optical Flow Algorithms

COLLEGE OF COMPUTING
KHON KAEN UNIVERSITY

Deep NN algorithms are having an impact on practical optical flow estimation

- A number of deep NN algorithms for optical flow have been proposed in recent years, starting around 2017
- Early algorithms used pure NN architectures (mostly CNN) proved impractical:
  - Lower accuracy when compared to conventional algorithms, e.g. TV-L1
  - Too slow for real-time use
- The PWC-Net algorithm, Sun, et. al., 2018, combines conventional and deep NN methods
  - PWC-Net is accuracy completive with conventional methods
  - Can be faster than TV-L1!

Deep Optical Flow Algorithms

Deep NN algorithms are having an impact on practical optical flow estimation

- Comparison of NN optical flow algorithms, [Sun, et. al., 2018]

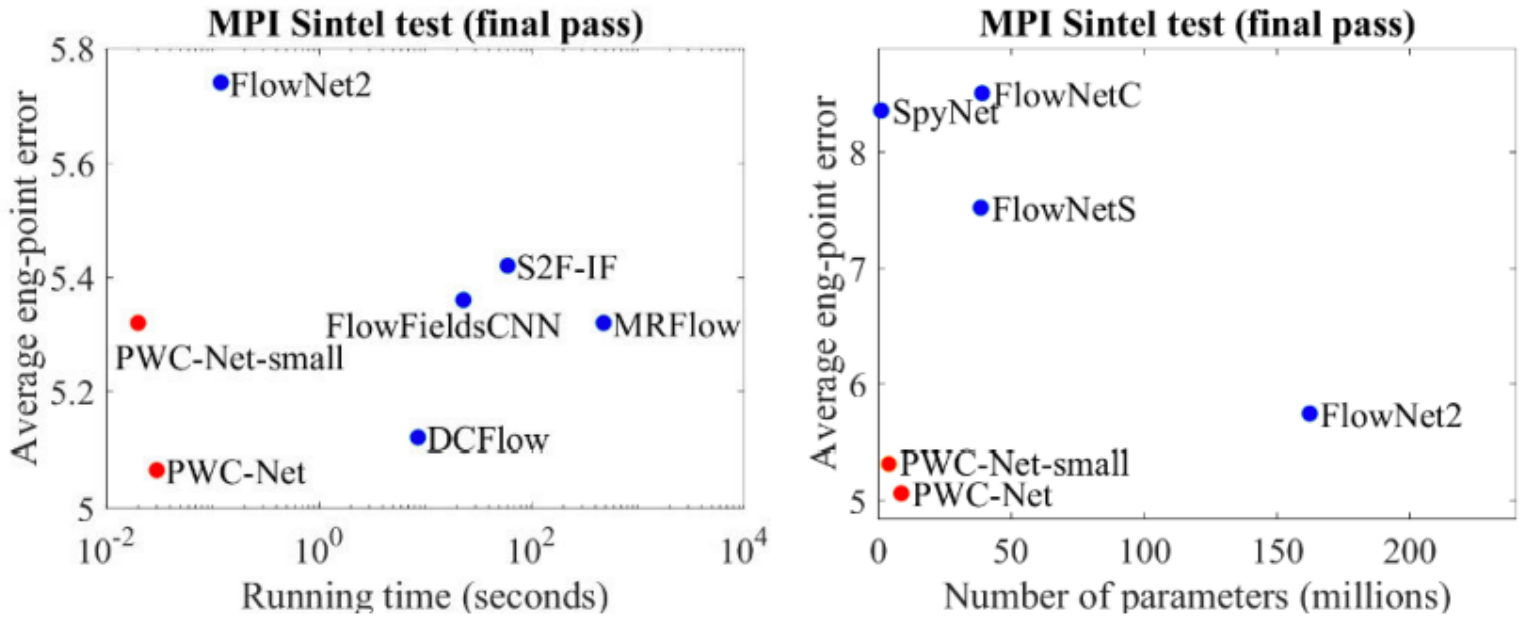
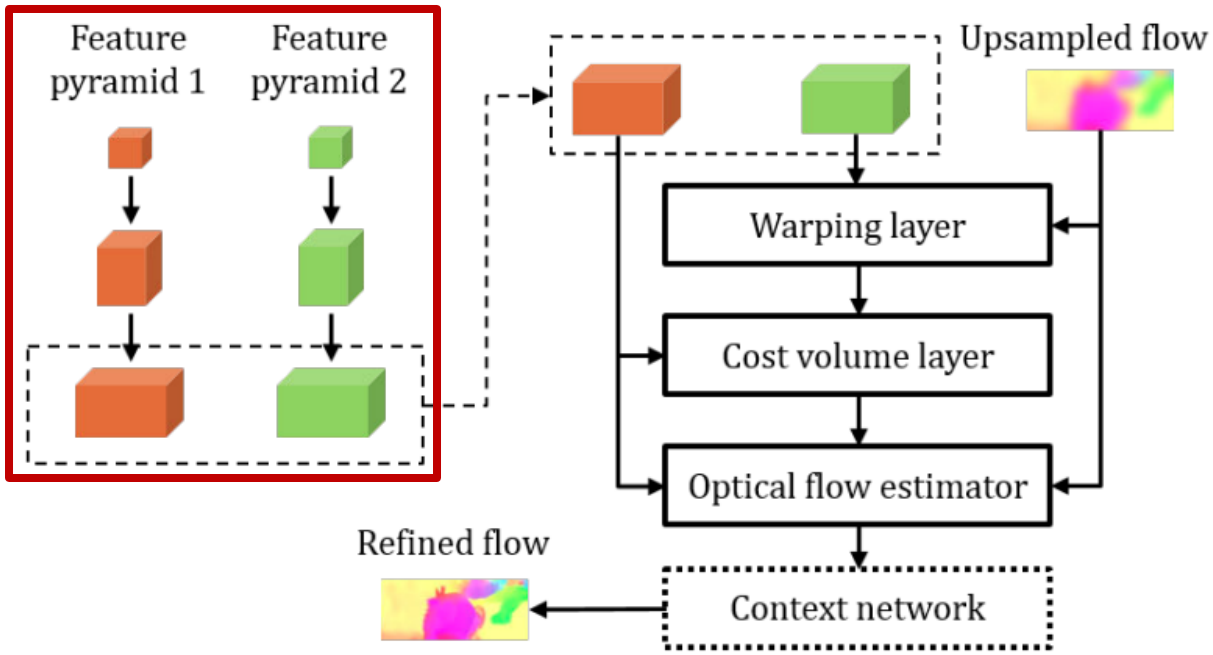


Figure 1. Left: PWC-Net outperforms all published methods on the MPI Sintel final pass benchmark in both accuracy and running time. Right: among existing end-to-end CNN models for flow, PWC-Net reaches the best balance between accuracy and size.

Deep Optical Flow Algorithms

The PWC-Net Algorithm

- Start with image pyramids
- Optimal **image pyramid** layers are learned as CNNs

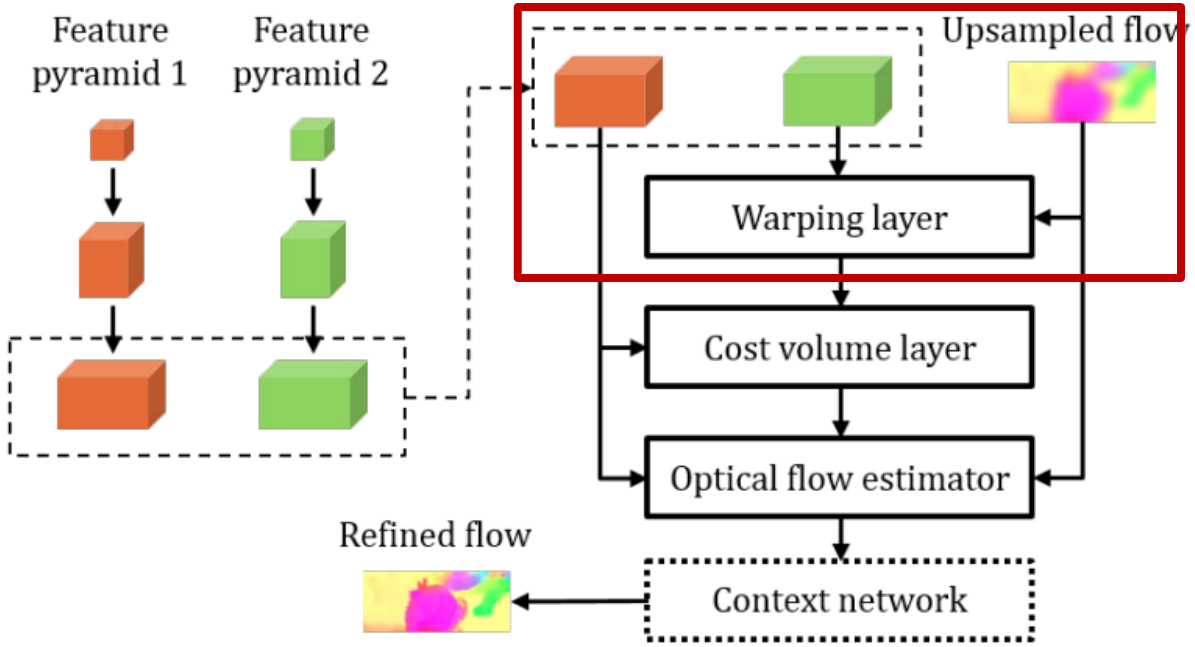


Credit Sun, et. al., 2018

Deep Optical Flow Algorithms

The PWC-Net Algorithm

- Image from pyramid 2 are warped with conventional algorithm



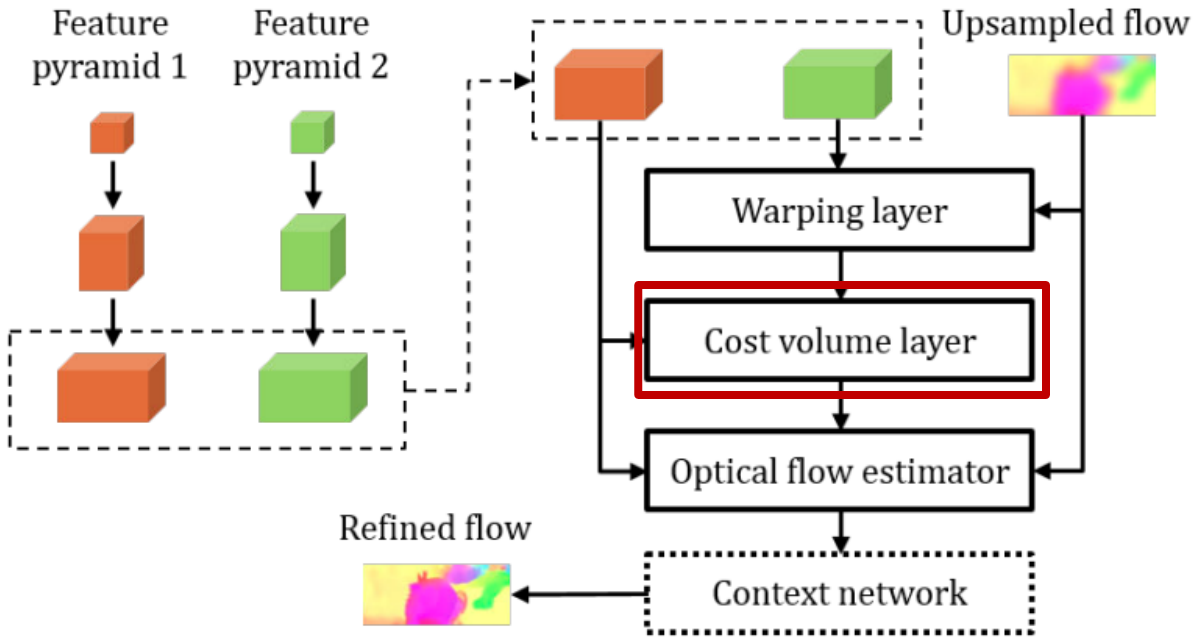
Credit Sun, et. al., 2018

Deep Optical Flow Algorithms

The PWC-Net Algorithm

- **Cost volume** layer maintains array of feature **matching costs** of $n \times m$ values at pyramid level, *l*
- Matching cost is the **correlation** between the feature (pixel) vector of the first image $c_1^l(x_1)$ and the feature vector of the warped second image
- Filter to keep only **near neighbor values** with upper bound *d* to limit computation

$$|x_1 - x_2|_\infty \leq d$$

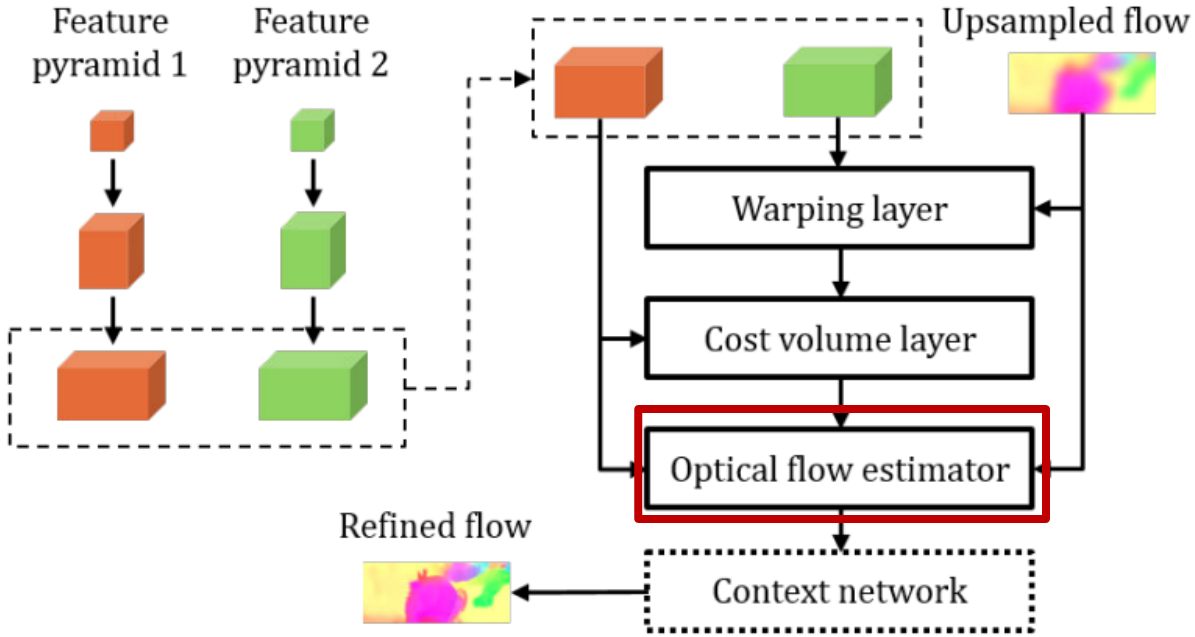


Credit Sun, et. al., 2018

Deep Optical Flow Algorithms

The PWC-Net Algorithm

- CNN used to estimate optical flow
- Training loss is least squares (Euclidian distance) between first image and warped second image, summed over levels of the pyramid

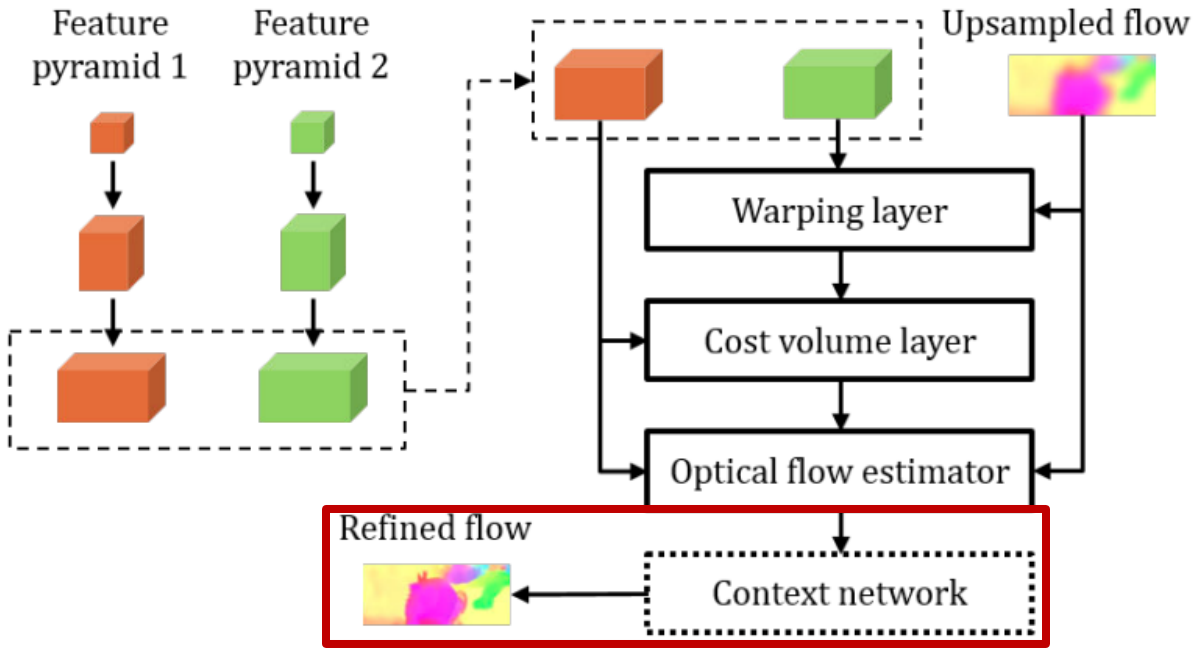


Credit Sun, et. al., 2018

Deep Optical Flow Algorithms

The PWC-Net Algorithm

- Conventional filtering used for post processing to improve continuity of flow field



Credit Sun, et. al., 2018

Deep Optical Flow Algorithms

PWC-Net algorithm tracking image-to-image motion

Cred Figure 5. Results on KITTI 2015 *training* and *test* sets. Fine-tuning fixes large regions of errors and recovers sharp motion boundaries.

TensorFlow implementation

References for Datasets and Algorithms

COLLEGE OF COMPUTING
KHON KAEN UNIVERSITY

Some additional algorithms and algorithms can be found on the Papers with Code site.

- Optical flow algorithms and datasets
- Multi-object tracking algorithms and datasets; a related topic